Assignment 2: JavaScript (Building on Assignment 1)

Overview

This assignment has a written part (Part A) and three build questions. You do not start from scratch: each question upgrades one of the projects you built in Assignment 1 using JavaScript.

Question	Upgrade this project from Assignment 1
Question 1	Your portfolio website
Question 2	The HR portal (login, directory and onboarding form)
Question 3	The landing page for your local business

Copy your Assignment 1 folders into a new `Assignment2` folder and work on the copies. Keep your Assignment 1 submission untouched.

Be creative. The elements listed in each question are the minimum. Add your own ideas, your own messages and your own design touches so that your work looks and behaves differently from everyone else's. Submissions that look alike will be questioned.

Part A: Written questions

Answer in a file named `answers.txt`. Keep each answer short and in your own words. For output questions, write what appears on the console, in order.

1. What is the difference between `var`, `let` and `const`? Explain scope, redeclaration and hoisting with a small example for each.
2. What does `typeof` print for each of these: `4`, `"hello"`, `true`, `null`, `undefined`? Why is JavaScript called a dynamically typed language?
3. Explain `==` versus `===`. Then explain the difference between `||` and `??`, and say which one you would use for a default value when `0` is a valid input.
4. A server sends you a student with an id, a name, two phone numbers and an address that has a street and a city. Write it as a JSON object. Show how you read the second phone number and the city. What does `student?.address?.city` do when `address` is missing?
5. Given `var arr = [10, 45, 32, 76, 45, 61, 78];`, write one line each to get (a) the even numbers, (b) every number doubled, (c) the sum of all numbers.
6. Explain what a callback function is. Rewrite `function add(x, y) { return x + y; }` as an arrow function with a single statement.

7. What is the output order of this code, and why? `console.log("A"); setTimeout(() => console.log("B"), 1000); console.log("C");`
8. What is callback hell, and how do promises solve it? Name the three states of a promise and the two methods used to handle it.
9. Rewrite `sum(2, 3).then(r => console.log(r)).catch(e => console.log(e))` using `async` and `await` with `try` and `catch`. Why must the function be marked `async`?
10. What is the DOM? Name the method used to find an element by its id, and explain the difference between `innerHTML` and `value`.
11. What does `fetch(url)` return, and why do we need `await` twice, once for `fetch` and once for `response.json()`? What does `response.status` tell you?
12. What is a regular expression, and why is it better than many `if` statements for checking a password rule?

Question 1: Upgrade your portfolio

Brief. Recruiters do not just read a portfolio, they click around in it. Make your portfolio from Assignment 1 interactive. The structure and design stay as they are. The JavaScript is what you add.

Home page: welcome and theme

- **Welcome box.** An input for the visitor's name and a button. On click, show "Welcome, <name>!" in a heading using a template literal. If the input is empty, show a friendly message instead. Use `getElementById`, `value` and `innerHTML`.
- **Light and dark mode.** A button that switches the page between a light and a dark theme by changing `style.backgroundColor` and `style.color`. The button text changes between "Dark mode" and "Light mode" each time, using `if` and `else`.
- **Show and hide.** A button that hides and shows your skills list by changing `style.display`. The button text changes between "Hide" and "Show".
- **GitHub repositories.** A button that loads your public repositories with `fetch` from `https://api.github.com/users/<your-username>/repos`. Use `async` and `await`, check `response.status`, read the data with `response.json()`, and list the repository names by creating elements with `document.createElement` and `appendChild`. Show an error message in red if the request fails. If you do not have a GitHub account, use `https://jsonplaceholder.typicode.com/posts` and list the titles instead.

Projects page: data-driven table and cards

- **Data, not hard-coding.** Remove the project rows you typed by hand. Store at least 5 projects as objects in an array, each with `id`, `title`, `tech` (an array), `description`, `link` and, for some projects only, `demoLink`.

- **Classes.** Write a `Project` class with a constructor and a method such as `getSummary()`. Write a `FeaturedProject` class that `extends` it, calls `super()` and adds a `badge` property. Create your projects as objects of these classes.
- **Build the table with a loop.** For every project, create a row and its cells with `createElement`, fill them using destructuring, and add them to the `tbody` with `appendChild`. Where `demoLink` is missing, show "Not available" using `?.` and `??`.
- **Top two cards.** Build the two featured project cards the same way and show each badge on its card.
- **Search and filter.** A search box that filters by title and a dropdown that filters by technology, using `filter` with an arrow function. Show "Showing N of M projects" using a template literal. Use `map` once and `reduce` once somewhere in your code, for example `map` to build a list of titles and `reduce` to count the technologies you have used in total.
- **Loading from a pretend server.** Write a function `loadProjects()` that returns a promise. It waits 1.5 seconds with `setTimeout`, then resolves with your array, or rejects with a message if the array is empty. The page shows "Loading projects..." first. Handle this promise with `async`, `await`, `try` and `catch`. Handle one other promise in your portfolio with `.then()` and `.catch()`.

Contact page: form validation

- Name is required. Email must match a regular expression. Phone must be exactly 10 digits, checked with a regular expression. The message must be at least 20 characters long.
- Show each error in red in a `<span class="error-span">` next to its field.
- Name and email are checked while typing, using `onkeyup`. All fields are checked when the form is submitted, using `onsubmit="return validate()"`. If any field is wrong, the form is not submitted.
- When everything is valid, show a thank-you message with the visitor's name.

Question 2: Upgrade the HR portal

Brief. TechNova HR liked the three pages from Assignment 1 and now wants them to work like a real application: check what employees type, and show employee data that comes from a server instead of being typed by hand.

Login page (`login.html`)

- **Show and hide password.** A button next to the password field that switches the input between `type="password"` and `type="text"` using `setAttribute`. The button text changes between "Show" and "Hide".

- **Username and password checks.** The username is required. The password must match a regular expression: at least 8 characters with one uppercase letter, one lowercase letter, one number and one special character.
- Show the errors in red next to each field. Check the password while the user types (`onkeyup`) and everything again on submit (`onsubmit="return validate()"`).

Directory page (`directory.html`)

- **Data from a server.** Remove the rows you typed by hand. A button "Load employees" fetches the list from `https://jsonplaceholder.typicode.com/users` using `fetch`, `async`, `await` and `try/catch`. Print `response.status` in the console. If the request fails, show an error message on the page.
- **Build the table with a loop.** For each employee, create a row with `createElement`, fill it using destructuring (`id`, `name`, `username`, `email`, `phone`) and add it to the `tbody`.
- **Total.** Below the table, show "Total employees: N" using a template literal. It must update whenever a row is added or removed.
- **Search with a choice.** A dropdown to choose what to search by (Name, Email or Username) and a search box. Keep the current search in an object built with a computed key, for example `{[field]: text}`, and use `filter` to show matching employees.
- **Add and remove.** A small form to add an employee (name and department). The new employee is created as an object using shorthand properties, and you extend a loaded record with the spread operator, for example `{...employee, department}`. Every row has a Remove button that deletes the row with `remove()`.

Onboarding page (`onboarding.html`)

- **Validation.** Full name is required. The official email must match a regular expression. Highest qualification and department must be chosen, not left on the first empty option. Show each error in red next to its field and check again on submit.
- **Onboarding steps.** When the form is valid, do not send it to the server. Show these steps one after another: "Creating your account..." (1 second), "Sending welcome email..." (2 seconds), "Assigning your laptop..." (1.5 seconds), then "Welcome aboard, `<name>`!". Write each step as a function that returns a promise, and chain them with `.then()` so that they run in order. Return `false` from the submit handler so that the page does not reload.
- If any step can fail, for example when the name is shorter than 3 letters, reject the promise and show the message with `.catch()`.

Question 3: Order calculator for your landing page

Brief. Your business owner is happy with the landing page and now wants customers to place orders on it. Add an order section to the landing page from Assignment 1 and make it work. Use your own business's menu, plans or services.

- **Menu from data.** Store at least 5 items as objects in an array, each with `id`, `name` and `price`. Build the dropdown options with a loop using `createElement` and `setAttribute("value", ...)`.
- **Add to order.** A dropdown for the item, an input for the quantity and an "Add to order" button. Read the quantity with `parseInt`. If it is not a positive number, show an error message. Otherwise add the item to a `cart` array as an object and show it in an order list built with `createElement` and `appendChild`.
- **Remove.** Every line in the order list has a Remove button that deletes the line from the page and from the `cart` array.
- **Delivery charge.** A dropdown for the delivery area. Use a `switch` with `case`, `break` and `default` to set the delivery charge for each area.
- **Discount.** Use an `if`, `else if` ladder to give a discount that depends on the subtotal, for example 5%, 10% and 15% at three levels you choose. Use the conditional `? :` operator to show "1 item" or "2 items" correctly.
- **Bill.** Calculate the subtotal with `reduce`. Show subtotal, discount, delivery charge and final total in one message made with a template literal. The bill updates every time the order changes.
- **Your own callback.** Write one function of your own that takes another function as an argument, for example `applyRule(amount, rule)`. Call it once with an anonymous function and once with an arrow function.
- **Place the order.** A "Place order" button runs a function that returns a promise. It waits 2 seconds with `setTimeout`, then resolves with an order number, or rejects with a message if the cart is empty. Show "Placing your order..." while waiting. Handle the promise with `async`, `await`, `try` and `catch`.
- **Special offer.** A banner that appears 3 seconds after the page loads, using `setTimeout`, with a short offer message of your own.

Folder structure and rules

Use the same folders as Assignment 1, with the JavaScript added. Broken paths count as errors.

```
<YourName>_Assignment2/
├── answers.txt
├── portfolio/
│   ├── index.html
```

```
│   ├── projects.html
│   ├── contact.html
│   ├── style.css
│   ├── images/
│   └── js/
├── hr-portal/
│   ├── login.html
│   ├── directory.html
│   ├── onboarding.html
│   ├── style.css
│   └── js/
└── landing-page/
    ├── landing.html
    ├── landing.css
    ├── images/
    └── js/
```

Rules

- Start from your Assignment 1 files. Do not rebuild the pages from scratch.
- Write your JavaScript inside `<script>` tags at the end of the page, or in a file inside the `js/` folder linked with `<script src="js/main.js"></script>`.
- Use `let` and `const` by default. Use `var` only where a question asks you to show how it behaves, with a comment.
- Use `onclick`, `onkeyup` and `onsubmit` in your HTML to connect buttons and fields to your functions.
- Give every function a meaningful name and a short comment saying what it does.
- No JavaScript libraries or frameworks (no jQuery, no React). Use plain JavaScript only.
- Write your own code. Copying another student's code is treated as plagiarism.
- Open the browser console while testing. Your pages must run without red errors.
- Test everything in a browser before submitting, including what happens when a field is empty or a request fails.

Bonus (optional)

Attempt these only after finishing everything above. Each one goes beyond what has been taught so far, so you will need to research it.

- Use the `Date` object to change the greeting on your portfolio according to the time of day (Good morning, Good afternoon, Good evening).
- Use `localStorage` to remember the visitor's chosen theme, so the portfolio opens in the same mode next time.
- Use `sort` to let the visitor order the projects table by title.
- Replace the `onclick` attributes on one page with `addEventListener` in your script.